Zerava — Web App Architecture & MVP Implementation Document

0. Purpose of This Document

This document defines **exactly how the Zerava MVP web app is built**, how the frontend and backend interact, and how work is split between contributors.

This is not aspirational. This is executable.

If something is unclear after reading this, it is a failure of the document.

---

1. Product Recap (Context Lock)

Zerava is a **fast, high-signal security scanning web app** for startups and small teams.

The MVP answers one question:

> "Is my public-facing website or API dangerously insecure right now?"

Constraints: - Scan completes in under 60 seconds - Results are understandable by non-security founders - No noise, no fear tactics, no enterprise bloat

---

2. High-Level Architecture Overview

Architecture Style: - API-first backend - Stateless frontend - Job-based asynchronous scanning

Tech Stack: - Frontend: React - Backend: Flask (Python) - Scanning: Custom Python security tools - Communication: JSON over HTTPS

The frontend never performs security logic. The backend never controls UI behavior.

---

3. Frontend Responsibilities (React)

The frontend exists to: - Collect user input - Show progress clearly - Present results cleanly and calmly

3.1 Core Pages

Landing / Scan Page - URL or API endpoint input field - Single primary CTA: "Run Scan" - Clear explanation of what will happen

Scan Progress Page - Status indicator (Queued → Running → Complete) - No fake loading animations - Honest timing indicators

Results Page - Security score (0–100) - Risk level labels (Critical / Medium / Low) - Plain-language explanations - Clear fix steps

---

4. Frontend Visual Direction

The frontend must visually align with the Zerava brand.

Visual Rules: - Minimal layouts - Calm spacing - No aggressive colors - Accent color used sparingly

UI Principles: - One primary action per screen - No clutter - No security jargon

The UI should feel like reading a clear report, not using a hacking tool.

---

5. Backend Responsibilities (Flask)

The backend is the brain of Zerava.

It is responsible for: - Accepting scan requests - Running security checks - Scoring risks - Generating explanations - Returning structured results

The backend never formats UI. It returns **data, not opinions**.

---

6. Scanning Engine Design

6.1 Scanning Philosophy

- Signal over coverage
- High confidence findings only
- Low false positives

6.2 Initial Scan Modules

MVP scan modules include: - HTTPS enforcement - SSL/TLS configuration checks - Security headers analysis - Open ports detection - Basic OWASP Top 10 indicators

Each module: - Runs independently - Returns structured findings - Has clear severity mapping

---

7. Asynchronous Job Model

Scanning is asynchronous.

Why: - Prevents request timeouts - Allows progress tracking - Scales better

Flow: 1. Frontend submits scan request 2. Backend creates scan job 3. Job status stored in memory or datastore 4. Scan executes in background 5. Results stored and exposed via API

---

8. Frontend–Backend API Contract

8.1 Scan Request

Endpoint: POST /api/scan

Request Body:

```
{
  "target": "https://example.com",
  "type": "website" | "api"
}
```

Response:

```
{
  "scan_id": "uuid",
  "status": "queued"
}
```

8.2 Scan Status

Endpoint: GET /api/scan/{scan_id}/status

Response:

```
{
  "status": "queued" | "running" | "completed" | "failed"
}
```

8.3 Scan Results

Endpoint: GET /api/scan/{scan_id}/results

Response:

```
{
  "score": 78,
  "risk_level": "medium",
  "findings": [
    {
      "severity": "critical",
      "title": "Missing Security Headers",
      "business_impact": "Increases risk of data exposure",
      "fix": "Add recommended headers to your server config"
    }
  ]
}
```

---

9. Security Score Calculation

The score is calculated based on: - Severity weight - Number of high-impact findings - Exposure relevance

Rules: - Scores change predictably - No dramatic swings - Score explanation is always available

The score builds trust. It is not a gimmick.

---

10. Data Model (Conceptual)

Scan Object: - id - target - type - status - timestamp

Finding Object: - severity - category - description - fix

Report Object: - scan_id - score - findings

---

11. Directory Structure

Frontend (React)

```
frontend/
  src/
    components/
    pages/
    services/
      api.js
    styles/
```

Backend (Flask)

```
backend/
  app/
    routes/
    scanners/
    scoring/
    models/
  run.py
```

---

12. Collaboration Workflow

Frontend Developer: - Owns UI - Uses API contract only - Does not infer backend behavior

Backend Developer: - Owns scan logic - Guarantees API responses - Does not adjust logic for UI convenience

Communication happens through: - This document - API schemas

No guessing. No silent changes.

---

13. MVP Completion Criteria

The MVP is complete when: - A user can run a scan without guidance - Results are understandable without security knowledge - Findings feel relevant and trustworthy - The system works consistently

Anything else is iteration.

---

14. Execution Lock

Once this document is agreed on: - No architectural debates - No scope expansion - No redesign mid-build

Execution follows clarity.

Zerava's web app is designed to be **calm, fast, and trustworthy**. Everything else is noise.